

```
!pip install catboost lightgbm -q
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import recall_score, precision_score
from catboost import CatBoostClassifier
from xgboost import XGBClassifier
```

```
from google.colab import files
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 169df72b552611f1.zip to 169df72b552611f1 (5).zip

```
import zipfile

with zipfile.ZipFile("169df72b552611f1.zip", 'r') as zip_ref:
    zip_ref.extractall("dataset")
```

```
import zipfile

with zipfile.ZipFile("/content/169df72b552611f1.zip", 'r') as zip_ref:
    zip_ref.extractall("/content/dataset")
```

```
train = pd.read_csv("/content/dataset/dataset/train.csv")
test = pd.read_csv("/content/dataset/dataset/test.csv")

print(train.shape)
print(test.shape)
```

```
(1352, 51)
(339, 50)
```

```
features = [col for col in train.columns if col not in ["CoilID", "Y"]]

train["sum_feat"] = train[features].sum(axis=1)
test["sum_feat"] = test[features].sum(axis=1)

train["mean_feat"] = train[features].mean(axis=1)
test["mean_feat"] = test[features].mean(axis=1)

train["std_feat"] = train[features].std(axis=1)
test["std_feat"] = test[features].std(axis=1)

train["max_feat"] = train[features].max(axis=1)
test["max_feat"] = test[features].max(axis=1)

train["min_feat"] = train[features].min(axis=1)
test["min_feat"] = test[features].min(axis=1)

train["range_feat"] = train["max_feat"] - train["min_feat"]
```

```

test["range_feat"] = test["max_feat"] - test["min_feat"]

train["median_feat"] = train[features].median(axis=1)
test["median_feat"] = test[features].median(axis=1)

train["skew_feat"] = train[features].skew(axis=1)
test["skew_feat"] = test[features].skew(axis=1)

train["kurt_feat"] = train[features].kurt(axis=1)
test["kurt_feat"] = test[features].kurt(axis=1)

train["q1_feat"] = train[features].quantile(0.25, axis=1)
test["q1_feat"] = test[features].quantile(0.25, axis=1)

train["q3_feat"] = train[features].quantile(0.75, axis=1)
test["q3_feat"] = test[features].quantile(0.75, axis=1)

```

```

X = train.drop(["CoilID", "Y"], axis=1)
y = train["Y"]

X_test = test.drop(["CoilID"], axis=1)

```

```

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

test_preds = np.zeros(len(X_test))
oof_preds = np.zeros(len(X))

```

```

for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):

    print(f"Fold {fold+1}")

    X_train = X.iloc[train_idx]
    y_train = y.iloc[train_idx]

    X_val = X.iloc[val_idx]
    y_val = y.iloc[val_idx]

    # =====
    # CATBOOST
    # =====

    cat_model = CatBoostClassifier(
        iterations=1200,
        learning_rate=0.03,
        depth=5,
        l2_leaf_reg=20,
        loss_function='Logloss',
        eval_metric='F1',
        auto_class_weights='Balanced',
        random_seed=42,
        verbose=200
    )

    cat_model.fit(
        X_train,

```

```

        y_train,
        eval_set=(X_val, y_val),
        use_best_model=True
    )

# =====
# XGBOOST
# =====

xgb_model = XGBClassifier(
    n_estimators=1200,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=5,
    subsample=0.7,
    colsample_bytree=0.7,
    gamma=2,
    reg_alpha=2,
    reg_lambda=3,
    scale_pos_weight=12,
    eval_metric='logloss',
    random_state=42
)

xgb_model.fit(X_train, y_train)

# =====
# VALIDATION PREDICTIONS
# =====

cat_val = cat_model.predict_proba(X_val)[:, 1]
xgb_val = xgb_model.predict_proba(X_val)[:, 1]

ensemble_val = (
    0.85 * cat_val +
    0.15 * xgb_val
)

# =====
# TEST PREDICTIONS
# =====

cat_test = cat_model.predict_proba(X_test)[:, 1]
xgb_test = xgb_model.predict_proba(X_test)[:, 1]

ensemble_test = (
    0.85 * cat_test +
    0.15 * xgb_test
)

test_preds += ensemble_test / 5

oof_preds[val_idx] = ensemble_val

```

Fold 1

0:	learn: 0.7963082	test: 0.7737256	best: 0.7737256 (0)	total: 25.1ms	remainin
200:	learn: 0.9657116	test: 0.8412783	best: 0.9040674 (90)	total: 3.21s	remainin
400:	learn: 0.9961240	test: 0.6214610	best: 0.9040674 (90)	total: 5.28s	remainin
600:	learn: 1.0000000	test: 0.5492627	best: 0.9040674 (90)	total: 7.34s	remainin
800:	learn: 1.0000000	test: 0.4649480	best: 0.9040674 (90)	total: 9.41s	remainin
1000:	learn: 1.0000000	test: 0.4663453	best: 0.9040674 (90)	total: 11.5s	remainin
1199:	learn: 1.0000000	test: 0.4663453	best: 0.9040674 (90)	total: 14.4s	remainin

```
bestTest = 0.9040674252
```

```
bestIteration = 90
```

```
Shrink model to first 91 iterations.
```

```
Fold 2
```

0:	learn: 0.8238591	test: 0.6462891	best: 0.6462891 (0)	total: 12.6ms	remainin
200:	learn: 0.9721304	test: 0.6190006	best: 0.8029131 (13)	total: 2.08s	remainin
400:	learn: 0.9937228	test: 0.3369168	best: 0.8029131 (13)	total: 4.12s	remainin
600:	learn: 0.9990291	test: 0.3417658	best: 0.8029131 (13)	total: 6.18s	remainin
800:	learn: 1.0000000	test: 0.3437448	best: 0.8029131 (13)	total: 8.24s	remainin
1000:	learn: 1.0000000	test: 0.3447429	best: 0.8029131 (13)	total: 11.7s	remainin
1199:	learn: 1.0000000	test: 0.3447429	best: 0.8029131 (13)	total: 13.8s	remainin

```
bestTest = 0.802913064
```

```
bestIteration = 13
```

```
Shrink model to first 14 iterations.
```

```
Fold 3
```

0:	learn: 0.8182371	test: 0.7291408	best: 0.7291408 (0)	total: 12.5ms	remainin
200:	learn: 0.9744318	test: 0.6544909	best: 0.8105553 (40)	total: 2.08s	remainin
400:	learn: 0.9956459	test: 0.4501312	best: 0.8105553 (40)	total: 4.14s	remainin
600:	learn: 0.9990291	test: 0.4527555	best: 0.8105553 (40)	total: 6.31s	remainin
800:	learn: 1.0000000	test: 0.2569609	best: 0.8105553 (40)	total: 9.75s	remainin
1000:	learn: 1.0000000	test: 0.2595372	best: 0.8105553 (40)	total: 11.8s	remainin
1199:	learn: 1.0000000	test: 0.2595372	best: 0.8105553 (40)	total: 13.9s	remainin

```
bestTest = 0.8105553369
```

```
bestIteration = 40
```

```
Shrink model to first 41 iterations.
```

```
Fold 4
```

0:	learn: 0.8163772	test: 0.7445192	best: 0.7445192 (0)	total: 12.6ms	remainin
200:	learn: 0.9689266	test: 0.7212196	best: 0.8560408 (101)	total: 2.1s	remainin
400:	learn: 0.9975763	test: 0.5356306	best: 0.8560408 (101)	total: 4.61s	remainin
600:	learn: 1.0000000	test: 0.5431226	best: 0.8560408 (101)	total: 7.7s	remainin
800:	learn: 1.0000000	test: 0.5477192	best: 0.8560408 (101)	total: 9.77s	remainin
1000:	learn: 1.0000000	test: 0.4649534	best: 0.8560408 (101)	total: 11.8s	remainin
1199:	learn: 1.0000000	test: 0.4649534	best: 0.8560408 (101)	total: 13.9s	remainin

```
bestTest = 0.8560408396
```

```
bestIteration = 101
```

```
Shrink model to first 102 iterations.
```

```
Fold 5
```

0:	learn: 0.8332195	test: 0.7605603	best: 0.7605603 (0)	total: 13ms	remainin
200:	learn: 0.9621318	test: 0.8305085	best: 0.8305085 (194)	total: 2.99s	remainin
400:	learn: 0.9946834	test: 0.7994648	best: 0.8468090 (374)	total: 5.6s	remainin
600:	learn: 1.0000000	test: 0.6181727	best: 0.8468090 (374)	total: 7.68s	remainin

```
for t in [0.14, 0.16, 0.18, 0.20]:
```

```
    preds = (oof_preds > t).astype(int)
```

```
    recall = recall_score(y, preds)
```

```
    precision = precision_score(y, preds)
```

```
    print("Threshold:", t)
```

```
    print("Recall:", recall)
```

```
    print("Precision:", precision)
```

```
    print("-----")
```

```
Threshold: 0.14
```

```
Recall: 0.9242424242424242
```

```
Precision: 0.07860824742268041
-----
Threshold: 0.16
Recall: 0.9242424242424242
Precision: 0.08379120879120878
-----
Threshold: 0.18
Recall: 0.9242424242424242
Precision: 0.08827785817655572
-----
Threshold: 0.2
Recall: 0.9242424242424242
Precision: 0.0915915915915916
-----
```

```
final_predictions = (test_preds > 0.13).astype(int)

print(pd.Series(final_predictions).value_counts())
```

```
1    188
0    151
Name: count, dtype: int64
```

```
submission = pd.DataFrame({
    "CoilID": test["CoilID"],
    "Y": final_predictions
})

submission.to_csv("final_submission.csv", index=False)

submission.head()
```

	CoilID	Y
0	711	1
1	1542	0
2	1232	0
3	600	0
4	1087	1

```
from google.colab import files
files.download("final_submission.csv")
```

